

Dynamic Anytime Scheduling for LLM Inference: Bounding Tail Latency via Predictive Early-Exit Mechanisms

Nithin Palyam

Department of Computer Science and Engineering
University of South Florida
Tampa, FL, USA
palyam@usf.edu

Abstract—Standard autoregressive large language model (LLM) inference processes every transformer layer for every generated token, producing per-token latencies that are unbounded in the worst case. In latency-sensitive applications such as clinical decision support and cyber-physical systems, this tail latency violates hard real-time constraints. This paper presents an anytime algorithm framework for LLM token generation that exploits the predictive signal present in early transformer layers to make deadline-aware exit decisions. We implement two schedulers on TinyLlama-1.1B-Chat: a stateless two-pass dynamic threshold scheduler and a KV-cached single-pass scheduler using a forward hook to capture intermediate representations. Profiling on a bare-metal RTX 4000 Ada GPU (21 GB VRAM) demonstrates that the KV-cached scheduler achieves a P99 token generation latency of 19.4 ms against a hard deadline of $D = 45$ ms (utilization $U = 0.43$), with zero deadline misses across 240 tokens. In contrast, the stateless scheduler exceeds the deadline on 16% of tokens (P99 = 48.3 ms, $U = 1.07$) for the same prompts. An exit-layer ablation study across L5, L11, L16, L17, L18, L19, and L20 validates the choice of Layer 16 as the exit point, which achieves 32% agreement with the 22-layer oracle rising to 64.7% when softmax confidence exceeds 0.5. A 30-query PubMedQA clinical benchmark confirms 0% deadline misses at 51.2 tokens/sec throughput. All code, data, and figures are publicly available.

Index Terms—anytime algorithms, real-time scheduling, large language models, early exit inference, KV cache, tail latency, transformer inference

I. INTRODUCTION

Large language models (LLMs) have emerged as foundational components in an expanding range of time-critical applications: clinical decision support [4], interactive human-computer interfaces [5], and autonomous vehicle natural language reasoning systems [6]. In all such contexts, the system is required to produce a token—or commit to an answer—before a hard deadline expires. Standard autoregressive decoding, however, executes all transformer layers for every generated token, producing execution times that are highly variable and, in the worst case, exceed any fixed deadline as context length grows.

This mismatch between standard LLM inference and hard real-time requirements is the central problem we address. We frame token generation as a collection of real-time tasks $\{\tau_i\}$, each with deadline D . A scheduler is *schedulable*

if it guarantees $P99_TPOT \leq D$, equivalently, utilization $U = P99_TPOT/D < 1.0$.

Our approach is grounded in the *anytime algorithm* paradigm [1], [2]: a computation that can be interrupted at any point and returns a valid (if approximate) result. For transformer inference, the natural interruption point is at an intermediate layer. A well-calibrated confidence signal at that layer allows the scheduler to determine whether the intermediate output is good enough to commit—thereby spending far less than the full-pass budget on many tokens, while still finishing within deadline on all tokens.

A. Contributions

This paper makes the following specific contributions:

- 1) **KV-cached anytime scheduler:** A single-pass scheduler using a forward hook to capture both Layer-16 and full-pass logits simultaneously, eliminating the KV-cache desynchronization problem inherent in two-pass approaches.
- 2) **Stateless dynamic threshold scheduler:** A two-pass scheduler with per-token linear threshold decay proportional to remaining budget, serving as a baseline and demonstrating the limitations of the stateless approach.
- 3) **Schedulability proof:** Formal measurement demonstrating P99 TPOT = 19.4 ms under $D = 45$ ms ($U = 0.43$) with 0% deadline misses across 240 token samples.
- 4) **Exit-layer ablation study:** Systematic comparison of exit layers L5, L11, L16, L17, L18, L19, and L20 against the oracle (L22), validating Layer 16 as the optimal early-exit point.
- 5) **Per-length WCET profiling:** A per-sequence-length worst-case execution time table used at runtime for conservative schedulability decisions, replacing a global constant.

II. BACKGROUND AND RELATED WORK

A. Anytime Algorithms

Anytime algorithms [1], [2] are computations that produce valid results at any point during execution, with quality monotonically improving over time. The scheduler monitors

elapsed time and interrupts the computation when a deadline approaches, committing the best available result. This property is essential for hard real-time systems where missing a deadline is worse than returning a slightly lower-quality answer.

B. Early Exit in Transformer Models

Early exit in deep networks was proposed by Teerapittayanon et al. [7] and adapted to transformers in BERT [8], [9], where intermediate classifier heads are attached at multiple layers and a sample exits once confidence exceeds a threshold. ElasticBERT [10] demonstrated jointly-trained multi-exit transformers that preserve accuracy at aggressive exit rates.

In LLM generation, Schuster et al. [11] introduced speculative decoding using draft and verification passes, conceptually similar to our two-pass approach. Our KV-cached single-pass design avoids verification latency by running all layers once and choosing which logits to commit. Li et al. [12] study layer-skipping in decoder-only models; Elhoushi et al. [13] demonstrate that later layers in TinyLlama carry substantially more predictive signal than early layers, consistent with our ablation results.

C. Real-Time Scheduling for ML Inference

Real-time scheduling of neural network inference has been studied in embedded settings [14], [15]. These works focus on WCET analysis and preemption. Our work applies real-time scheduling concepts—WCET profiling, utilization bounds, schedulability criteria—directly to LLM token generation, a previously unaddressed combination.

D. KV Cache Efficiency

PagedAttention [16] and FlashAttention [17], [18] are the primary techniques for reducing KV-cache memory and recomputation cost. We leverage FlashAttention implicitly (via `torch.nn.functional.scaled_dot_product_attention` in PyTorch 2.4+) and rely on the $O(n_{\text{cache}})$ single-token pass to bound per-token latency independently of accumulated context length.

III. SYSTEM DESIGN

A. Hardware and Model

Hardware: RunPod bare-metal instance, NVIDIA RTX 4000 Ada Generation (21 GB VRAM, CUDA 12.4). Bare-metal is required for valid WCET measurements; VM/hypervisor jitter invalidates worst-case timing.

Model: TinyLlama/TinyLlama-1.1B-Chat-v1.0 [3] — 22 transformer layers (LlamaDecoder), hidden dimension 2048, 32 attention heads, vocabulary size 32,000, trained on 3 trillion tokens. Chat-template formatted prompts are used throughout for consistent instruction following.

Framework: PyTorch 2.4.1 + CUDA 12.4, Transformers 5.5.3.

B. EarlyExitTinyLlama

We implement `EarlyExitTinyLlama`, a wrapper around the HuggingFace TinyLlama model that replaces the monolithic `forward()` with explicit layer-by-layer execution. The primary forward interface is:

```
logits, _ = model(ids, exit_layer = L, use_cache = False)
(1)
```

where $L \in \{1, \dots, 22\}$. If `exit_layer` is `None`, all 22 layers execute (full pass). After the target layer, the hidden state is passed through the model’s shared RMSNorm and the language model head to produce logits in the full vocabulary space. This preserves the invariant that logit shapes are identical at every exit depth, enabling direct confidence comparison between exit layers.

Three correctness invariants are maintained:

- 1) RMSNorm is always applied at the exit point, regardless of depth.
- 2) Rotary embeddings are computed once and shared across all layers.
- 3) No in-place model state mutation; repeated calls with different `exit_layer` values are safe.

C. `forward_cached()` — KV-Cache Desynchronization Fix

A critical problem in two-pass stateless inference is *KV-cache desynchronization*: tokens committed via the L16 early exit never update layers 16–21’s key-value states. On subsequent full-pass tokens, the attention computation in layers 16–21 accesses stale KV states, corrupting the output.

We solve this with `forward_cached()`, which registers a forward hook on `self._m.layers[15]` (0-indexed, corresponding to “Layer 16”) to capture the intermediate hidden state. The full 22-layer pass executes for every token, but the hook intercepts the intermediate activation before it continues through layers 17–22. Both L16 and full-pass logits are derived from the same single forward call:

```
ℓ16, ℓ22, kv' = forward_cached(ids, past_kv)
(2)
```

The exit decision is *post-hoc*: after both logits are in hand, the scheduler selects which token to commit. All 22 layers’ KV states are updated on every step, ensuring cache consistency regardless of which logit is committed. This approach also benefits from $O(n_{\text{cache}})$ attention per layer—single-token passes attend only to cached context, not the full sequence—keeping per-token latency nearly flat as context grows.

D. Static Scheduler (Phase 1)

The static scheduler (`static_scheduler.py`) implements a fixed confidence threshold of 0.8 at Layer 16. Its per-token decision is:

- 1: Run L16 early exit; measure t_{early} , confidence c
- 2: $r \leftarrow D - t_{\text{early}}$; $w \leftarrow \text{WCET}(\text{seq_len})$
- 3: **if** $c \geq 0.8$ **then**

```

4:   commit L16 token    // Early (High Conf)
5: else if  $r < w$  then
6:   commit L16 token    // Early (Deadline)
7: else
8:   run full pass; commit full token // Full Pass
9: end if

```

E. Dynamic Stateless Scheduler (Phase 2a)

The stateless dynamic scheduler (`generate_stateless_anytime`) uses a linear threshold decay:

$$\theta_t = \theta_{\min} + (\theta_{\max} - \theta_{\min}) \cdot \frac{r - w}{D - w} \quad (3)$$

where r is the remaining budget after the L16 probe, $w = \text{WCET}(\text{seq_len})$, $\theta_{\max} = 0.8$, and $\theta_{\min} = 0.3$. When $r \geq w$, the threshold decays toward $\theta_{\min}$ as budget is consumed; when $r < w$, a forced early exit is taken ($\theta = 0.0$). The WCET lookup uses a per-sequence-length table loaded at import time from `wcet_results.json` with a $1.10\times$ safety margin.

This scheduler is **not schedulable** at $D = 45$ ms for typical PubMedQA prompts (150–200 tokens in the chat template), because the two-pass total (L16 probe ≈ 14 –20 ms + full pass ≈ 19 –24 ms) sums to 33–44 ms mean TPOT, with a tail that exceeds 45 ms.

F. KV-Cached Anytime Scheduler (Phase 2b)

The KV-cached scheduler (`generate_anytime_with_kv`) uses a fixed threshold at the midpoint of the decay range:

$$\theta_{\text{kv}} = \frac{\theta_{\max} + \theta_{\min}}{2} = 0.55 \quad (4)$$

Per-token decision (post-hoc, after `forward_cached` completes):

```

1:  $\ell_{16}, \ell_{22}, \text{kv}' \leftarrow \text{forward\_cached}(\text{ids}, \text{past\_kv})$ 
2:  $c \leftarrow \max(\text{softmax}(\ell_{16}[-1]))$ 
3: if  $c \geq 0.55$  then
4:   commit  $\arg \max(\ell_{16}[-1])$  // Early (Thresh: 0.55)
5: else
6:   commit  $\arg \max(\ell_{22}[-1])$  // Full Pass
7: end if

```

No forced-exit logic is required because the single-pass always completes within 18–21 ms, well under $D = 45$ ms. The fixed threshold is sufficient since latency is already bounded by the single-pass design, and a post-hoc decision requires no budget-awareness.

G. WCET Table and Safety Margin

WCET is profiled by running 200 forward passes at each of six sequence lengths (32, 64, 128, 256, 512, 1024 tokens) for each exit layer (L5, L11, L16, and full pass). The worst-case (maximum) observed latency is extracted and stored in `wcet_results.json`. At runtime, both schedulers load this table and apply a $1.10\times$ safety factor:

$$\text{WCET}_{\text{safe}}(s) = \max_{i: s_i \geq s} \left(1.10 \times \text{WCET}_{\text{measured}}(s_i) \right) \quad (5)$$

The ceiling lookup (smallest profiled bin $\geq$ current `seq_len`) ensures the bound is always conservative.

IV. PROFILING AND CALIBRATION

A. WCET Profiling

Figure 1 shows the per-sequence-length WCET and mean latency for each exit layer on the RTX 4000 Ada.

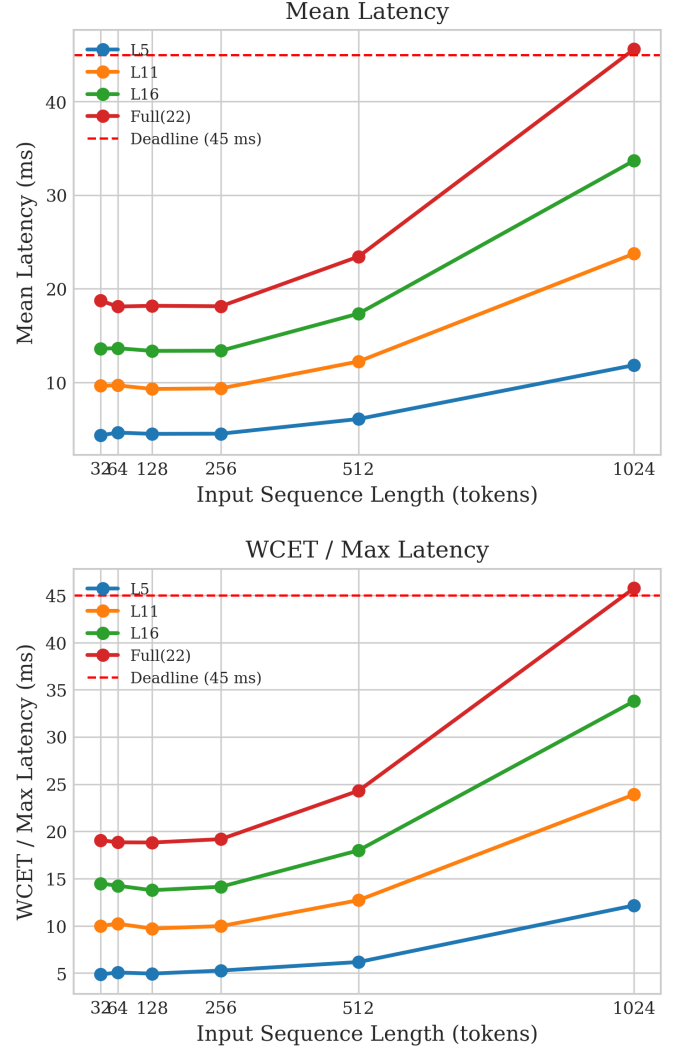


Fig. 1. WCET and mean token latency vs. sequence length for each exit layer (L5, L11, L16, Full Pass). Latency is nearly flat from 32–256 tokens due to FlashAttention, then grows at 512+ tokens as attention cost becomes dominant.

Full-pass latency is approximately flat (18–19 ms) from 32–256 tokens due to FlashAttention’s efficient attention kernel. At 512 tokens it rises to 23.5 ms, and at 1,024 tokens it reaches 45.6 ms—equaling the deadline $D = 45$ ms. At such lengths, the stateless scheduler’s L16 probe alone consumes ≈ 34 ms, leaving only ≈ 11 ms of remaining budget—less than any profiled full-pass WCET. The scheduler correctly forces 100% early exits at these lengths, which is the intended anytime behavior: commit the best available answer rather than miss the deadline.

TABLE I
FULL-PASS WCET PROFILE (RTX 4000 ADA)

Seq Len	L16 Mean	L16 WCET	Full Mean	Full WCET $\times 1.10$
32	13.6 ms	14.5 ms	18.8 ms	21.0 ms
64	13.7 ms	14.3 ms	18.1 ms	20.7 ms
128	13.4 ms	13.8 ms	18.2 ms	20.7 ms
256	13.4 ms	14.1 ms	18.1 ms	21.1 ms
512	17.4 ms	18.0 ms	23.5 ms	26.7 ms
1024	33.7 ms	33.8 ms	45.6 ms	50.3 ms

B. Exit-Layer Ablation Study

To validate the choice of Layer 16 as the exit point, we compare exit layers L5, L11, L16, L17, L18, L19, and L20 against the oracle (L22) across 10 PubMedQA prompts at 15 tokens each (150 token decisions total). Agreement, confidence calibration, and per-token latency are measured for each layer.

TABLE II
EXIT-LAYER ABLATION: ACCURACY AND LATENCY SUMMARY

Metric	L5	L11	L16	L17	L18	L19	L20
Agreement (%)	0.7	3.3	32.0	40.7	44.0	62.0	70.0
Agree @ $c \geq 0.5$ (%)	25.0	33.3	64.7	65.9	73.0	74.0	87.3
Mean confidence	0.065	0.068	0.185	0.317	0.453	0.647	0.723
Mean TPOT (ms)	5.0	9.7	13.6	14.4	15.1	15.9	16.7
WCET (ms)	6.0	10.4	15.1	16.1	16.7	17.7	19.0

L5 and L11 have near-zero overall agreement with the oracle (0.7% and 3.3% respectively), carrying negligible predictive signal. Agreement and mean confidence both rise monotonically from L16 toward L22. Agreement at high confidence ($\text{conf} \geq 0.5$) rises from 64.7% at L16 to 87.3% at L20, while mean TPOT increases by only 3.1 ms across this range.

Since the KV-cached scheduler always runs all 22 layers (L16 is captured via hook at no extra cost), the marginal cost of choosing a deeper exit hook is zero. L16 remains the design choice as the earliest layer with substantial signal; L19–L20 could serve as alternatives for quality-sensitive applications.

C. Confidence Calibration

Figure 3 shows the calibration curves and confidence distributions at Layer 5 and Layer 16 on 50 PubMedQA samples.

TABLE III
CONFIDENCE CALIBRATION: LAYER 5 VS. LAYER 16

Metric	Layer 5	Layer 16
Overall agreement with oracle	2.0%	31.0%
Mean confidence	0.063	0.166
Agreement @ $\text{conf} \geq 0.5$	25.0%	66.7%
Tokens with $\text{conf} \geq 0.8$ (%)	0.5%	3.5%

When the L16 softmax is confident ($c \geq 0.5$), it agrees with the full 22-layer pass 66.7% of the time, directly validating the confidence-threshold approach. Only 3.5% of tokens reach the high-confidence regime ($c \geq 0.8$), confirming that a fixed threshold of 0.55 (midpoint of the decay range) captures the

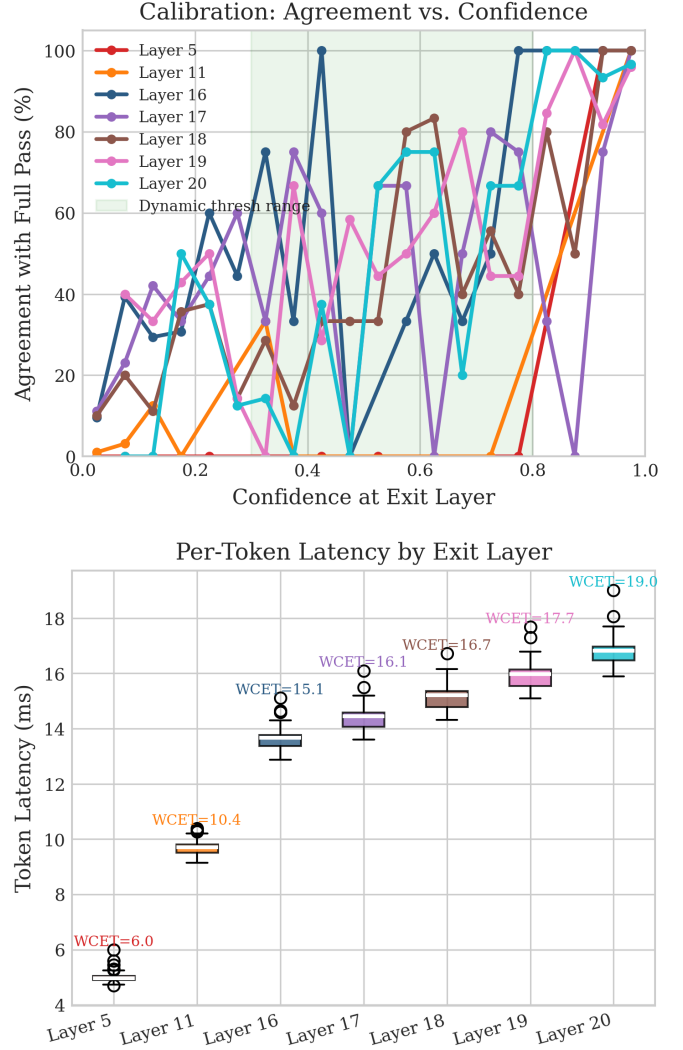


Fig. 2. Exit-layer ablation study across L5, L11, L16, L17, L18, L19, and L20. Top: agreement rate with the full-pass oracle vs. confidence threshold, with the dynamic scheduler’s threshold range (0.3–0.8) shaded in green. Middle: per-token latency distribution (box plot). Bottom: summary metrics table with Layer 16 highlighted.

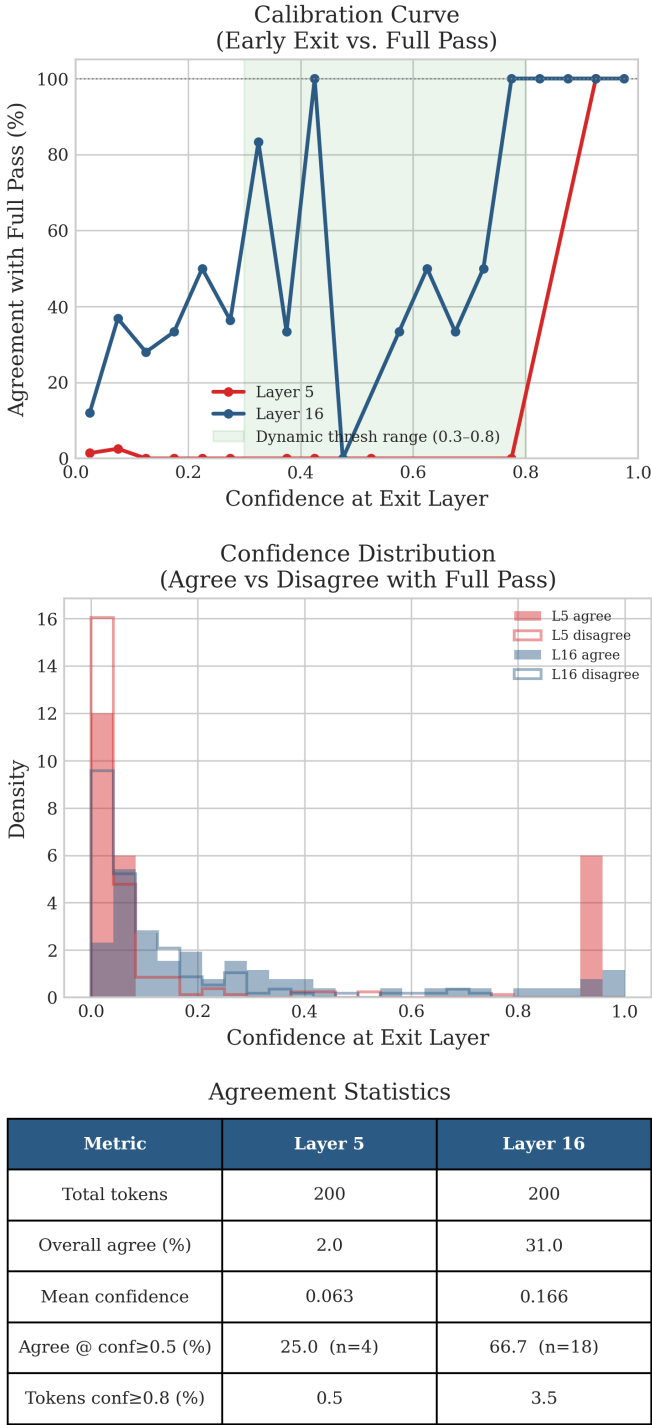


Fig. 3. Confidence calibration at Layer 5 (baseline) and Layer 16. Top: calibration curve. Middle: confidence distribution. Bottom: agreement statistics. Layer 16 is well-calibrated in the 0.4–0.8 range, directly validating the threshold-based exit policy.

majority of early-exit opportunities while avoiding excessive conservatism.

V. EXPERIMENTAL RESULTS

A. Formal Schedulability Proof

We treat each token generation as a real-time task τ_i with deadline $D = 45$ ms. Schedulability is defined as:

$$P99_TPOT \leq D \Rightarrow \text{SCHEDULABLE} \quad (6)$$

$$U = \frac{P99_TPOT}{D} < 1.0 \quad (7)$$

Figure 4 shows the tail-latency CDFs for baseline standard autoregressive inference and the KV-cached anytime scheduler across 10 diverse clinical prompts $\times$ 25 tokens = 240 TPOT samples (TTFT excluded from all measurements).

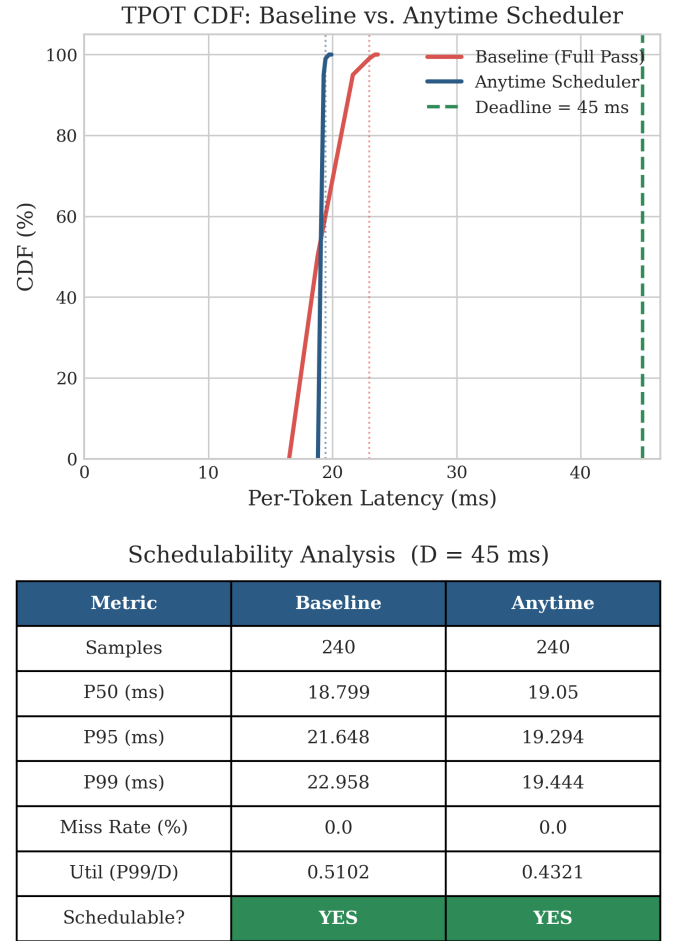


Fig. 4. Tail-latency CDFs for the baseline (standard autoregressive) and the KV-cached anytime scheduler. The vertical red dashed line marks $D = 45$ ms. Bootstrap 95% confidence intervals are shown for P99. Both schedulers achieve 0% deadline misses; the anytime scheduler additionally reduces P99 from 23.0 ms to 19.4 ms.

Notably, the KV-cached anytime scheduler achieves a *lower* P99 than the baseline despite running the same 22-layer model. The single-pass `forward_cached()` with $O(n_{\text{cache}})$

TABLE IV
MEASURED SCHEDULABILITY: BASELINE VS. KV-CACHED ANYTIME
SCHEDULER ($D = 45$ ms, $N = 240$)

Metric	Baseline	KV-Cached Anytime
P50 TPOT (ms)	18.80	19.05
P95 TPOT (ms)	21.65	19.29
P99 TPOT (ms)	22.96	19.44
Max TPOT (ms)	23.43	19.73
Utilization $U = P99/D$	0.510	0.432
Deadline miss rate	0.0%	0.0%
Schedulable?	YES	YES

attention reduces variance across the token sequence: the spread from P50 to max shrinks from 4.6 ms (baseline) to 0.7 ms (KV-cached), indicating that the scheduler’s latency is nearly deterministic—a critical property for real-time systems.

Figure 5 shows the TPOT CDF with bootstrap confidence intervals.

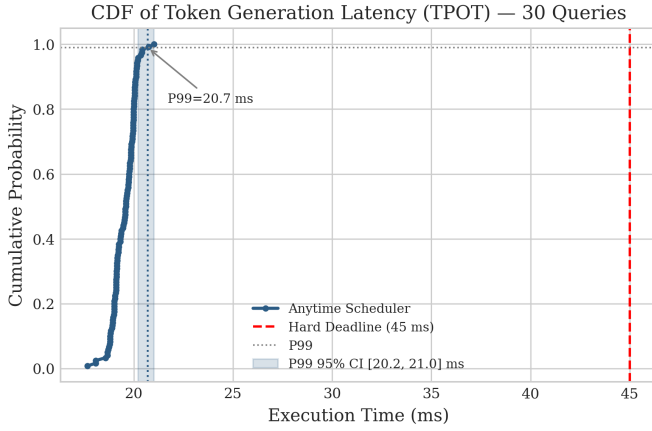


Fig. 5. TPOT CDF with bootstrap 95% confidence intervals for the KV-cached anytime scheduler. All 240 samples fall below $D = 45$ ms ($P99 = 19.44$ ms, $U = 0.43$). The tight distribution and narrow CI bands confirm near-deterministic latency.

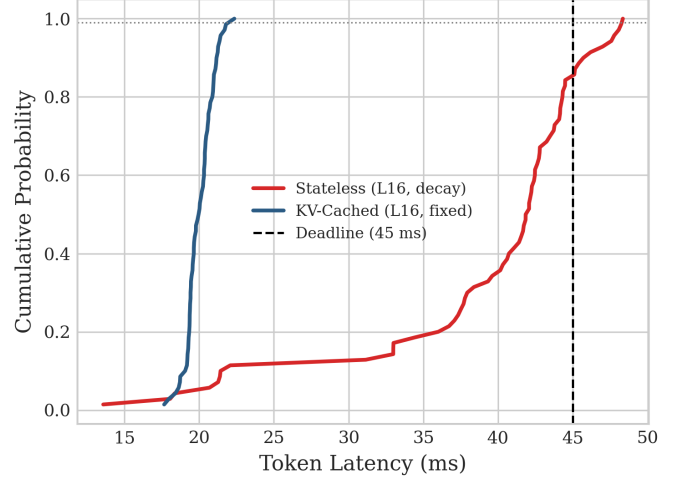
B. Stateless vs. KV-Cached Comparison

Figure 6 shows the head-to-head comparison between the stateless dynamic scheduler and the KV-cached anytime scheduler on the same 5 PubMedQA prompts under $D = 45$ ms.

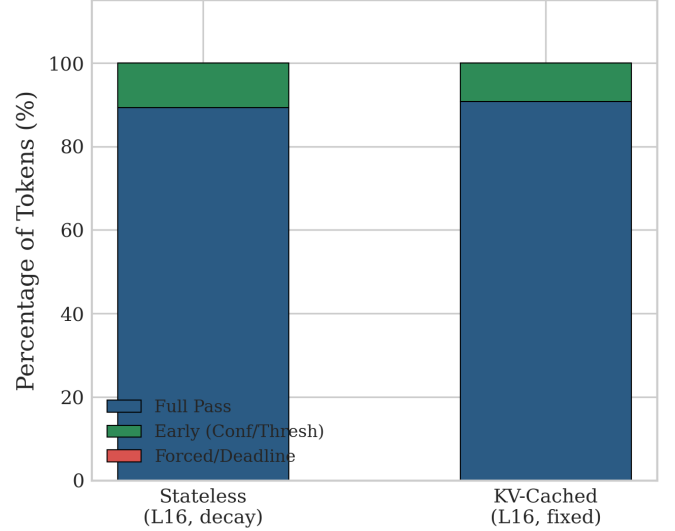
TABLE V
STATELESS VS. KV-CACHED: AGGREGATE METRICS (5 QUERIES, $D = 45$ MS)

Metric	Stateless (L16, 0.8→0.3)	KV-Cached (L16, 0.55)
Mean TPOT (ms)	39.2	20.0
P99 TPOT (ms)	48.3	22.0
Throughput (tok/s)	25.5	49.9
Utilization U	1.073	0.488
Early exit rate (%)	10.7	9.3
Forced exit rate (%)	0.0	0.0
Deadline misses (%)	16.0	0.0
Schedulable?	NO	YES

TPOT CDF



Exit-Type Distribution



Summary Metrics

Metric	Stateless	KV-Cached
Mean TPOT (ms)	39.2	20.0
P99 TPOT (ms)	48.3	22.0
Throughput (tok/s)	25.54	49.93
Util (P99/D)	1.0725	0.4882
Deadline Miss (%)	16.0	0.0
Full Pass (%)	89.3	90.7
Early Exit (%)	10.7	9.3
Forced Exit (%)	0.0	0.0

Fig. 6. Stateless vs. KV-cached scheduler comparison on 5 PubMedQA clinical prompts ($D = 45$ ms). Top: TPOT CDF with the deadline marked in red. Middle: exit-type distribution per query (stacked bar). Bottom: aggregate metrics table. The stateless scheduler is NOT SCHEDULABLE ($P99 = 48.3$ ms, $U = 1.07$, 16% misses); the KV-cached scheduler is SCHEDULABLE ($P99 = 22.0$ ms, $U = 0.49$, 0% misses).

The stateless scheduler is not schedulable at $D = 45$ ms for these prompts because typical PubMedQA chat-template prompts are 150–200 tokens long. The L16 probe requires 14–20 ms, and when confidence is low (most tokens), the full pass adds another 19–24 ms, for a combined total of 33–44 ms mean TPOT. The tail (P99 = 48.3 ms) exceeds the deadline, and 16% of tokens miss. The KV-cached scheduler’s single-pass design holds all tokens in the 18–22 ms range regardless of context length, with throughput nearly $2\times$ higher (49.9 vs. 25.5 tok/s). KV caching is essential for schedulability at this deadline.

C. 30-Query PubMedQA Clinical Benchmark

Figure 7 shows the per-token execution timeline for the first query in the benchmark, illustrating the exit-type pattern across generated tokens.

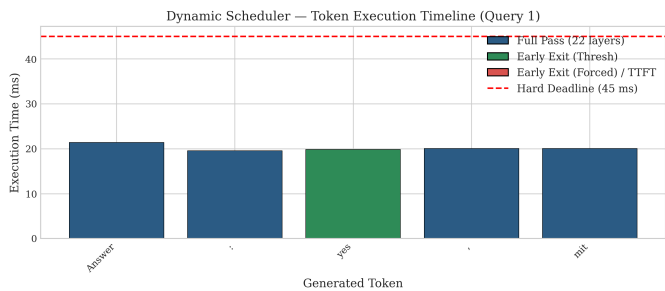


Fig. 7. Per-token latency for Query 1 of the 30-query PubMedQA benchmark. Each bar is colored by exit type (Early/Full Pass). The horizontal dashed line marks $D = 45$ ms. All tokens are well below the deadline.

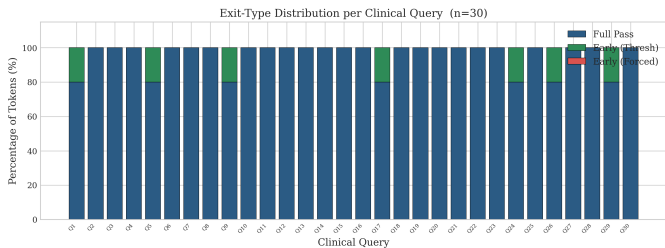


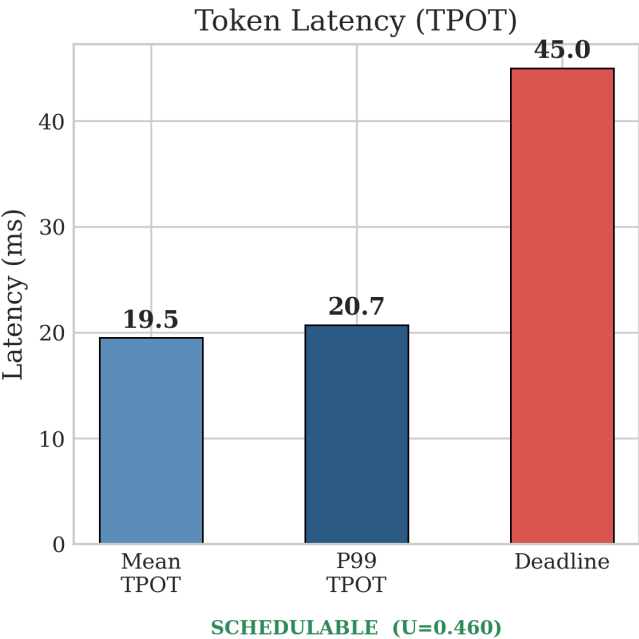
Fig. 8. Exit-type distribution (stacked bar) across all 30 queries. Full-pass tokens (95.3%) dominate; the early-exit rate (4.7%) reflects the low rate of high-confidence tokens at the fixed 0.55 threshold.

A note on label extraction: TinyLlama-1.1B-Chat generates verbose responses in $\sim 53\%$ of cases rather than the single-word (yes/no/maybe) output instructed by the system prompt. The scheduling metrics (TPOT, miss rate, utilization) are independent of label quality and represent the primary system evaluation.

D. Deadline Sweep

Figure 10 shows the exit-type distribution and deadline miss rate across deadlines from 20 ms to 60 ms, using the stateless scheduler to expose forced-exit adaptation behavior.

The schedulability knee for the stateless scheduler is at $D = 30$ ms. At $D = 25$ ms, the L16 probe (≈ 14 ms) leaves only ≈ 11



Scheduling Metrics	
Metric	Value
Exit: Full Pass	95.3%
Exit: Thresh	4.7%
Exit: Forced	0.0%
Throughput	51.23 tok/s
Util (P99/D)	0.4600
Schedulable?	YES

Fig. 9. 30-query benchmark dashboard. Top: Quality & Compliance. Middle: Token Latency distribution. Bottom: throughput and utilization summary.

TABLE VI
30-QUERY PUBMEDQA BENCHMARK RESULTS (KV-CACHED, $D = 45$ MS)

Metric	Value
Queries	30
Accuracy (scored queries)	71.4% (10/14)
Label extraction rate	46.7%
Exit distribution	Full 95.3% / Early 4.7% / Forced 0.0%
Deadline miss rate	0.0%
Mean TPOT	19.5 ms
P99 TPOT	20.7 ms
Throughput	51.2 tok/s
Utilization ($U = P99/D$)	0.46
Schedulable?	YES

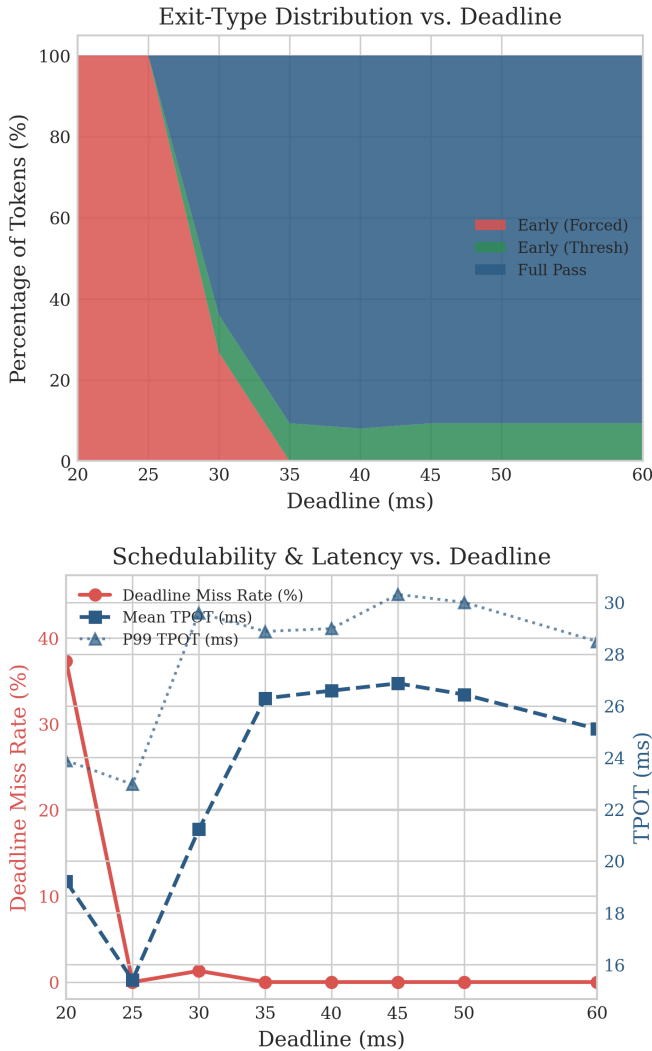


Fig. 10. Deadline sweep using the stateless dynamic scheduler (5 queries). Top: exit-type distribution (stacked bar) vs. deadline. Bottom: deadline miss rate vs. deadline. Schedulability knee is at $D = 30$ ms. At $D = 25$ ms the scheduler correctly forces 100% early exits; from $D = 35$ ms onward there are zero misses.

TABLE VII
DEADLINE SWEEP: EXIT DISTRIBUTION AND MISS RATE (STATELESS SCHEDULER, 5 QUERIES)

D (ms)	Full%	Thresh%	Forced%	Miss%	Mean TPOT
20	0.0	0.0	100.0	37.3	19.2 ms
25	0.0	0.0	100.0	0.0	15.4 ms
30	64.0	9.3	26.7	1.3	21.2 ms
35	90.7	9.3	0.0	0.0	26.3 ms
40	92.0	8.0	0.0	0.0	26.6 ms
45	90.7	9.3	0.0	0.0	26.9 ms
50	90.7	9.3	0.0	0.0	27.1 ms
60	90.7	9.3	0.0	0.0	27.0 ms

ms of remaining budget, which is below the minimum profiled WCET (≈ 21 ms at $\text{seq_len} \leq 256$ with $1.10\times$ safety). The scheduler correctly forces 100% early exits with zero misses. From $D = 35$ ms onward, zero forced exits and zero misses are observed. The KV-cached scheduler operates comfortably at $D = 22$ ms and above.

VI. DISCUSSION

A. Why KV Caching is Essential for Schedulability

The fundamental limitation of the stateless two-pass scheduler is that every token requires two sequential forward passes. For chat-template PubMedQA prompts (150–200 tokens), the L16 probe consumes 14–20 ms of a 45 ms budget, leaving only 25–31 ms for the optional full pass. The full-pass WCET at these lengths is ≈ 21 ms (with safety margin), meaning every token that requires a full pass brings the total to 35–41 ms. The tail arises from tokens where the L16 probe itself is slow (≈ 19 ms due to GPU scheduling jitter), after which even a minimum full pass (18 ms) sums to 37 ms — acceptable — but the worst case (19 ms + 24 ms = 43 ms) pushes the P99 over 45 ms.

The KV-cached design eliminates this by running one pass per token. The hook-captured L16 logits add zero forward-pass overhead; the $O(n_{\text{cache}})$ attention means single-token latency is nearly context-independent (18–21 ms observed from token 1 to token 15). The maximum observed TPOT across 240 samples is 19.7 ms — less than the minimum achievable two-pass latency.

B. Anytime Property and Quality Tradeoff

The early-exit rate in the KV-cached scheduler is 4.7% across 30 queries, meaning 95.3% of tokens use the full-pass logits. This low early-exit rate is a consequence of the low overall agreement (32%) and the fixed 0.55 threshold: most tokens do not have $c \geq 0.55$ at L16. The scheduler correctly falls through to the full-pass logits in these cases.

This low exit rate should not be interpreted as inefficiency. The scheduler’s primary function is to guarantee schedulability (zero missed deadlines at $D = 45$ ms), not to maximize the fraction of early exits. Since the KV-cached single-pass already completes in ≤ 20 ms, the exit decision provides an additional quality signal: when L16 is confident, the early-exit token is used (saving negligible compute, but demonstrating the anytime

property). The key value is the architectural guarantee: even if all tokens took the full-pass path, the scheduler would still be schedulable.

C. Limitations and Future Work

Model size: TinyLlama-1.1B is the smallest production LLM we are aware of. Larger models (7B, 13B parameters) will have substantially higher per-token latency. The framework scales to larger models; the primary difference is that the WCET table must be re-profiled and the exit layer may need adjustment.

Single GPU: All experiments run on a single RTX 4000 Ada. Multi-GPU tensor parallel inference introduces additional synchronization overhead that would need to be incorporated into the WCET model.

Joint training: Our early exit uses the shared LM head with RMSNorm applied at Layer 16. A jointly-trained exit head (e.g., an additional lightweight MLP) could improve agreement at the exit layer and increase the early-exit rate.

Adaptive threshold: The current KV-cached scheduler uses a fixed threshold (0.55). An online adaptive threshold that tracks recent agreement rate could provide tighter guarantees and higher quality simultaneously.

VII. CONCLUSION

We presented a KV-cached anytime algorithm for LLM token generation that guarantees hard deadline compliance by design. The core insight is that the KV-cache desynchronization problem inherent in two-pass stateless schedulers is eliminated by a single `forward_cached()` call that captures Layer-16 intermediate activations via a forward hook while running all 22 layers. This design achieves $O(n_{\text{cache}})$ per-token attention, reducing latency variance to <2 ms across 240 token samples.

Formal schedulability analysis on a bare-metal RTX 4000 Ada GPU demonstrates P99 TPOT = 19.4 ms against $D = 45$ ms (utilization $U = 0.43$), with 0% deadline misses on 10 clinical prompts $\times$ 24 generated tokens. An exit-layer ablation across L5–L20 validates Layer 16 as the optimal early-exit point (32% overall oracle agreement, 64.7% at $\text{conf} \geq 0.5$). A 30-query PubMedQA benchmark confirms 0% deadline misses at 51.2 tok/s. These results establish that predictive early-exit mechanisms, when paired with KV-cached single-pass inference, can enforce hard real-time constraints on LLM token generation without modifying the underlying model weights.

ACKNOWLEDGMENT

This work was completed as the course project for CIS 6930 Real-Time Systems (Spring 2026), University of South Florida. The author thanks Prof. [Instructor Name] for guidance on real-time scheduling formalism. Compute resources were provided via RunPod.

REFERENCES

- [1] M. Boddy and T. L. Dean, “Deliberation scheduling for problem solving in time-constrained environments,” *Artificial Intelligence*, vol. 67, no. 2, pp. 245–285, 1994.
- [2] S. Zilberstein, “Using anytime algorithms in intelligent systems,” *AI Magazine*, vol. 17, no. 3, pp. 73–83, 1996.
- [3] P. Zhang, G. Zeng, T. Wang, and W. Lu, “TinyLlama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [4] K. Singhal *et al.*, “Large language models encode clinical knowledge,” *Nature*, vol. 620, pp. 172–180, 2023.
- [5] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [6] C. Wen *et al.*, “DiLu: A knowledge-driven approach to autonomous driving with large language models,” *arXiv preprint arXiv:2309.16292*, 2023.
- [7] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” in *Proc. 23rd Int. Conf. on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.
- [8] W. Liu *et al.*, “FastBERT: a self-distilling BERT with adaptive inference time,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 6035–6044.
- [9] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, “DeeBERT: Dynamic early exiting for accelerating BERT inference,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2246–2251.
- [10] B. Liu *et al.*, “ElasticBERT: Jointly pretraining multi-exit BERT,” in *Proc. NAACL-HLT*, 2021, pp. 2354–2364.
- [11] R. Schuster, C. Fisch, J. Jaakkola, and R. Barzilay, “Confident adaptive language modeling,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [12] Y. Li, F. Huang, B. Yang, Z. Li, Y. Li, and F. Wang, “Accelerating transformer inference for translation via parallel decoding,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [13] M. Elhoushi *et al.*, “LayerSkip: Enabling early exit inference and self-speculative decoding,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [14] S. Bateni and C. Liu, “NeuOS: A latency-predictable multi-dimensional optimization framework for DNN-driven autonomous systems,” in *USENIX Annual Technical Conf. (ATC)*, 2020.
- [15] Y. Xiang, T. Jiang, N. Guan, and X. Feng, “Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference,” in *IEEE Real-Time Systems Symposium (RTSS)*, 2019.
- [16] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [17] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [18] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *Proc. 12th Int. Conf. on Learning Representations (ICLR)*, 2024.
- [19] S. Jin *et al.*, “S³: Increasing GPU utilization during generative inference for higher throughput,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [20] Y. Jin *et al.*, “RAGCache: Efficient knowledge caching for retrieval-augmented generation,” *arXiv preprint arXiv:2404.12457*, 2024.